

# Cache-Informed Prompting for Recursive Language Models

CS2640 Final Project Report

Fucheng Warren Zhu

May 2026

## 1. Motivation

Modern language models come with a 200K-token cache. The field calls it the *context window*. When it fills, the framework picks something to drop. The common choices are drop-oldest, summarize-and-collapse, and persist-to-filesystem-with-an-index. None of these came out of the cache literature. They came out of expedience.

This course covered the alternatives: LRU, LFU, S3-FIFO, SIEVE, admission control, prefetching. The question of this project is whether they help when stated as a prompt. Can we just tell the model to use a real eviction policy and have it work?

## 2. Background: Recursive Language Models

The substrate is the Recursive Language Model (RLM) of Zhang, Kraska, and Khattab [1]. RLM gives a root LM a Python REPL it can read and write across iterations. The user prompt does not enter the context window. It lives in the REPL as a variable `P`. The root LM sees only metadata about `P` (length, prefix) and writes code to peek into it, slice it, and pass slices to sub-LM calls. Variables persist across iterations. `del` evicts. The root LM runs until it emits `FINAL(answer)`.

The model is already the programmer of its own context. It decides what to keep, what to discard, and when to spawn sub-calls. The decisions are explicit code, not implicit attention. The analogy to a cache is literal here, not metaphorical.

## 3. Prior results

The RLM paper [1] reports large gains over context-compaction baselines: CodeQA 62 vs. 24 (base), BrowseComp+ 91 vs. 51 (CodeAct+BM25), OOLONG 56.5 vs. 46 (summary agent), OOLONG-Pairs 58 vs. 0.1 (base). Disabling sub-calls drops OOLONG-Pairs to 43.9, so recursion contributes. The full table is reproduced in Appendix G.

## 4. Methods

I ran four prompt arms against a vanilla RLM control on the same benchmarks the paper reports. Each arm appends a single hint block to the upstream RLM system prompt. The tone of every hint is suggestive, never directive, so the model can ignore it.

| Arm | Hint policy                                                                                   |
|-----|-----------------------------------------------------------------------------------------------|
| A0  | empty suffix (control)                                                                        |
| A1  | LRU. <i>Prefer to <b>del</b> the variable you have not touched recently.</i>                  |
| A3  | S3-FIFO. <i>Treat new variables as probation; promote on re-reference; track ghosts.</i>      |
| A4  | SIEVE. <i>Scan in creation order; evict the first one not re-referenced since last pass.</i>  |
| A6  | Theory primer: retention vs. reconstruction cost; LRU/LFU/S3-FIFO/SIEVE; admission filtering. |

Three non-RLM baselines ran alongside: `direct` (single Sonnet call with head+tail truncation), `summary_agent` (iterative chunk-and-summarize), and `codeact_bm25` (ReAct loop with BM25 over 400-token chunks). All eight methods used Claude Sonnet 4.6 over the Agent SDK against the same paired query samples per benchmark.

I attempted five benchmarks. Three made it into the results: S-NIAH (synthetic needle-in-haystack across 2K to 262K tokens), LongBench-v2 CodeQA (around 100K-token code repository understanding), and OOLONG-Pairs (32K-token pairwise reasoning). BrowseComp+ at the 1K-doc setting hit an Agent SDK stdin cap that no buffer-size workaround unblocked. OOLONG `trec_coarse` does not exist as a public split. Both postmortems are in Appendix B and C.

## 5. Results

Pilot N=3 to 10 per cell, paired across methods. Score is task-native (exact match for S-NIAH, letter exact for CodeQA, exact for OOLONG-Pairs). The Agent SDK injects a fixed ~16K-token Claude Code preamble into every call. This inflates absolute cost roughly 3-4x over the raw API but is constant across arms.

| Method                               | S-NIAH      | CodeQA      | OOLONG-Pairs |
|--------------------------------------|-------------|-------------|--------------|
| <code>direct</code>                  | <b>1.00</b> | <b>1.00</b> | <b>1.00</b>  |
| <code>summary_agent</code>           | 1.00        | 0.33        | 0.00         |
| <code>codeact_bm25</code>            | 0.43        | 0.33        | 0.00         |
| <b><code>rlm_a0 (vanilla)</code></b> | <b>1.00</b> | 0.50        | 0.25         |
| <code>rlm_a1 (LRU)</code>            | 0.67        | 0.00        | —            |
| <code>rlm_a3 (S3-FIFO)</code>        | 0.11        | 0.67        | 0.00         |
| <code>rlm_a4 (SIEVE)</code>          | 0.10        | —           | —            |
| <code>rlm_a6 (theory primer)</code>  | 0.30        | 1.00        | 0.00         |

Three patterns stand out.

First, eviction hints hurt retrieval. On S-NIAH, A3 (S3-FIFO) drops from vanilla 1.00 to 0.11. A4 (SIEVE) drops to 0.10. A1 (LRU) is gentler and lands at 0.67. Even the soft theory primer A6 drops to 0.30. The probable cause is mechanical. The REPL variable holding the haystack is exactly the kind of variable the hints tell the model to consider evicting. The model sometimes obliges, and the needle goes with it.

Second, the same hints flip sign on reasoning. On CodeQA, A3 lifts vanilla 0.50 to 0.67. A6 lifts it to 1.00. The hints help when intermediate state across iterations matters. They hurt when the task is pure retrieval. Hints act like a prior. They reward tasks where the prior is correct.

Third, vanilla RLM is not the strongest method on every task. On OOLONG-Pairs at 32K tokens, **direct** with truncation scores 1.00 because the prompt fits in context. RLM at 0.25 does not gain anything when there is no scale problem to solve. The treatment arms drop further from there, in the same direction as S-NIAH.

The CIs are wide. Nothing is significant at 95 percent (Appendix D). The directional signal is what motivates the hypotheses below.

## 6. The cost finding

Token consumption exceeded my forecast by several multiples. Three things drove the overrun. The Agent SDK preamble adds about 16K tokens to every call. Max-subscription quotas are token-weighted, so a single 100K-token CodeQA call drains more quota than twenty short S-NIAH calls. A per-cell hang in the runner cost me a 44-minute stall on the first night before I added a hard timeout.

A hardware cache miss costs microseconds. An LLM cache miss costs dollars and seconds. The gap is many orders of magnitude. That gap suggests an LLM cache should lean on conservative retention and on admission filtering before items enter the context window. I did not test this directly, but it is the design instinct the cost numbers point toward.

## 7. Conclusion and what comes next

Teaching an LLM cache theory through prompt hints is not free. On the workload I tested most heavily (retrieval), the hints make failures more likely by nudging the model to evict the answer. On reasoning tasks they trend positive, but the sample is too small to be sure. Vanilla RLM was the strongest cache-aware option I tested.

I think the policy may belong in the runtime rather than the prompt. A reference-counted `SHOW_VARS()`, an admission filter on variable creation, and a runtime that enforces an S3-FIFO discipline over the REPL would let the LM see the bookkeeping rather than carry it. The next experiment I would run is an A-prime arm where the RLM runtime tracks access counts and the LM sees them, with the policy enforced by code rather than by hint.

A second open question is KV-cache awareness. Policies that preserve the prefix keep KV reuse cheap. None of the arms in this study were KV-aware. There are potentially further efficiency gains and interesting work there.

## References

- [1] A. Zhang, T. Kraska, O. Khattab. “Recursive Language Models.” arXiv:2512.24601, 2026. [2] J. Yang et al. “FIFO Queues Are All You Need for Cache Eviction.” SOSP, 2023. [3] Y. Zhang et al. “SIEVE Is Simpler Than LRU.” NSDI, 2024. [4] Y. Bai et al. “LongBench v2.” 2024. [5] A. Yen et al. “OOLONG: Evaluating LLMs on Long Inputs.” 2024.

# Appendix

## A. Extended treatment arms (A7–A32)

Beyond the five arms reported in the main body, I ran 26 additional cache-policy hints overnight against S-NIAH. Most underperformed vanilla RLM. The catalog is below for transparency. Each arm is a single suffix block on the RLM system prompt with the same suggestive tone as A1–A6.

| Arm | Policy                           | S-NIAH (N=3) |
|-----|----------------------------------|--------------|
| A7  | ARC (Adaptive Replacement Cache) | 0.10         |
| A8  | TinyLFU                          | 0.10         |
| A9  | CLOCK                            | 0.00         |
| A10 | Two-Queue (2Q)                   | 0.10         |
| A11 | Segmented LRU                    | 0.00         |
| A12 | Victim cache                     | 0.00         |
| A13 | Bloom-filter admission           | 0.00         |
| A14 | Correlation-based prefetch       | 0.00         |
| A15 | Cache-oblivious                  | 0.10         |
| A16 | Content-defined chunking         | 0.10         |
| A17 | Write-back                       | 0.00         |
| A18 | MRU (Most Recently Used)         | 0.00         |
| A19 | Belady oracle (informative)      | 0.10         |
| A20 | LFU                              | 0.00         |
| A21 | LIRS                             | 0.00         |
| A22 | Hot-cold separation              | 0.00         |
| A23 | CLOCK-Pro                        | 0.00         |
| A26 | Cache partitioning (UCP)         | 1.00         |
| A27 | Line locking (real-time)         | 0.00         |
| A28 | Pin-clean (no meta-narrative)    | 1.00         |
| A29 | S3-FIFO structural labels only   | 0.00         |
| A30 | Single-sentence cache advice     | —            |
| A31 | “Orient then execute” structural | 0.50         |
| A32 | Plan-and-pin combined            | 1.00         |

The pattern is consistent with the main-body finding. Most directive eviction hints either match or hurt vanilla RLM on retrieval. The only arms that recover full score are A26 (cache partitioning, which structurally protects the haystack region), A28 (clean pin), and A32 (explicit plan-then-pin). All three share the same property: they tell the model to *protect* something, not to *evict* something.

## B. BrowseComp+ postmortem

Target was 1K-doc retrieval at 6M-11M tokens. The RLM paper handles this benchmark by chunking inside the REPL. My setup used the Agent SDK’s `claude_agent_sdk.query()` over OAuth, which spawns the Claude Code CLI as a subprocess and pipes the prompt over stdin. The CLI’s stdin line buffer caps below ~35K user tokens. A 395K-token BrowseComp+ prompt produces a bare `exit code 1` with no stderr. Raising `max_buffer_size` to 16 MB on the Python side did not help because the cap is on the CLI side. Raw Anthropic API would dodge this entirely but requires a real API key, and the project budget was OAuth-only. BrowseComp+ was dropped.

The dropped benchmark is also the one where the paper’s RLM lift is largest, so my evidence on that lift is missing.

## C. OOLONG (trec\_coarse) postmortem

The original plan included OOLONG `trec_coarse` at 131K context. The public `oolongbench/oolong-synth` HuggingFace dataset has no `trec_coarse` split. The paper presumably uses a private or unreleased version. I substituted OOLONG-Pairs (32K, semantic pairwise reasoning), which is in the same family and ships in the public dataset.

## D. Per-tier S-NIAH detail and bootstrap CIs

S-NIAH is run across context tiers from 2K to 262K tokens. The collapse pattern in §5 is uniform across tiers — the directive arms fail at small and large contexts alike, suggesting the failure mode is policy-driven rather than scale-driven.

Paired bootstrap CIs on score delta vs. `rlm_a0` (S-NIAH, N=3, 95%):

```
rlm_a1  Δ = -0.333  [-1.000, +0.000]
rlm_a3  Δ = -0.667  [-1.000, +0.000]
rlm_a4  Δ = -0.667  [-1.000, +0.000]
rlm_a6  Δ = +0.000  [+0.000, +0.000]
```

CIs are wide. None reach significance at 95%. Directional signal is consistent enough to motivate the headline claim but not strong enough to publish on.

## E. Behavioral analysis

Cluster-separation ratio between-arm vs. within-arm on the final-prediction feature vector was 1.00. The 1.5 threshold for “behaviorally distinct” is not met. Arms differ in score but not in the surface span of their final-answer string. The richer signal lives in the per-iteration REPL trace, which would require wiring `RLMLogger` into the runner. That is the first thing on the next-experiment list.

## F. Per-cell token usage breakdown

Available in `results/summary.csv`. Each row records benchmark, method, `query_id`, score, cost, duration, input tokens, output tokens, error. The cost field is Sonnet-only. The Haiku orchestrator the Agent SDK runs in the background is tracked separately and not summed into the main figure.

## G. Full prior-results table from Zhang+Kraska+Khattab 2026

GPT-5 row of Table 1 in the RLM paper. Accuracy in percent. Task lengths: CodeQA 23K-4.2M, BrowseComp+ 6M-11M, OOLONG 131K, OOLONG-Pairs 32K tokens.

| Method         | CodeQA      | BrowseComp+ (1K) | OOLONG      | OOLONG-Pairs |
|----------------|-------------|------------------|-------------|--------------|
| Base Model     | 24.0        | 0.0              | 44.0        | 0.1          |
| CodeAct + BM25 | 22.0        | 51.0             | 38.0        | 24.7         |
| Summary agent  | 58.0        | 70.5             | 46.0        | 0.1          |
| <b>RLM</b>     | <b>62.0</b> | <b>91.3</b>      | <b>56.5</b> | <b>58.0</b>  |

| Method             | CodeQA | BrowseComp+ (1K) | OOLONG | OOLONG-Pairs |
|--------------------|--------|------------------|--------|--------------|
| RLM (no sub-calls) | 58.0   | 88.0             | 36.0   | 43.9         |